700106 / 700120 demonstration

Introduction

The purpose of the final lab demonstration is:

- To assess the quality and quantity of the features you have chosen to implement. We'll work through each feature in the specification.
- To assess your knowledge and understanding of your implementation. We'll ask you a series of questions related to how and why you chose to implement a certain feature in the way you did.

We recognise that your solution may consist of a fairly large code base, some of which you may not have touched for several weeks prior to the demo. To simplify the process, we'll release a set of possible questions before the demonstrations, to enable you to prepare your answers.

Questions

1. Structure:
    a. Explain why you chose your data structure to hold your vertex data. Comment on the efficiency of the data structure for loading the data and also passing the data into Vulkan.
    b. Explain why you chose to implement your shader management system in the way you did. Comment on its efficiency and also programming ease by which different shaders can be assigned to different objects.
    c. Explain your scene management strategy and how you handle the different objects in the scene. Comment on why you did or did not use interfaces and/or abstract classes in your solution.
    d. Explain how you partitioned your solution to separate Vulkan logic from your game logic. Comment on the challenges created by your architecture if porting your solution to use DirectX.
    e. Explain how you implemented your model loader. Comment on the robustness of the approach, given that you were not required to implement the full OBJ specification.
    f. How have you represented and processed the geometry of the platforms and different objects created in your scene? Do they have the same format? Explain why your representations are appropriate.

2. Geometry Representation:
    a. How have you represented and processed the geometry of the desert plane, cacti, animals, and additional objects in your scene? Do these assets share a common format (e.g., OBJ), and why is this representation appropriate for your pipeline?
    b. For the object containing thousands of polygons, how did you optimise the loading, storage, and rendering of this object to maintain real time performance?
    c. Regarding the terrain mesh distortion, how did you generate elevation changes (hills/valleys) on the desert plane? Did you use procedural displacement,

heightmaps, or manual modelling? How does this affect lighting and shadow accuracy?

3. Shading Techniques:
    a. Which shading models have you implemented? How do these techniques enhance realism across different mesh types?
    b. For day/night lighting cycle, how did you implement the directional sun and moon lights? How do their intensities, colours, and directions evolve over time, and how does this influence the mood of the scene?
    c. If you added torch or localised lights (e.g., sparks from fire), how did you manage their attenuation, range, and interaction with nearby geometry?

4. Shadow Mapping Implementation:
    a. How have you implemented shadow mapping for the sun, moon, or other lights? What resolution, depth biasing, and filtering strategies did you use to reduce artefacts such as shadow acne or peter panning?
    b. If multiple lights cast shadows, how did you manage the performance cost? Did you use cascaded shadow maps, single pass techniques, or selective shadow casting?

5. Bump/Normal Mapping:
    a. Which bump mapping techniques (normal mapping, height mapping, parallax mapping, etc.) did you implement?
    b. How do they enhance the appearance of sand, cactus bark, or other surfaces?

6. Particle Systems and Dynamic Effects:
    a. How did you design and update particles for effects such as fire, snow, rain, dust, or sandstorms? Are your particles simulated on the CPU or GPU? What influenced your decision?

7. Deferred Rendering and Post Processing:
    a. If you implemented deferred rendering, which G buffers did you use (normals, albedo, depth, specular, etc.)? How does this architecture support your lighting model?
    b. What measurable performance benefits did deferred rendering provide compared to forward rendering in your scene?
    c. If you implemented MRT visualisation (optional feature), how did you expose the different buffers to the user, and how did this help debugging?
    d. If implemented HDR, how does HDR rendering improve the visual quality of bright desert sunlight or glowing fire? Which tone mapping operator did you choose and why?